A SHORT TALK IN FIVE PARTS

Selfie

Christoph Kirsch
University of Salzburg, Austria
Czech Technical University in Prague, Czechia

One file. 12,394 lines of C. It compiles itself, executes itself, and hosts itself.

IN THE FILE	
`starc`	a self-compiling compiler, C* to RISC-U
`mipster`	a self-executing emulator of RISC-U
`hypster`	a self-hosting hypervisor of RISC-U
`libcstar`	the library all three run on

No dependencies. No framework. No build system. Nothing hidden below it. You can read *all* of it — and in one semester you will have.

→ advances · n speaker notes · d light or dark

THE WHOLE TALK, UP FRONT

Three commands. Three kinds of self-reference.

```
// the compiler compiles its own source — twice
$ ./selfie -c selfie.c -o selfie1.m -m 2 -c selfie.c -o selfie2.m
$ diff -s selfie1.m selfie2.m
Files selfie1.m and selfie2.m are identical
```

```
// the emulator executes its own machine code
$ ./selfie -c selfie.c -o selfie.m -m 2 -l selfie.m -m 1
```

```
// the hypervisor hosts a virtual machine running the hypervisor
$ ./selfie -c selfie.c -o selfie.m -m 3 -l selfie.m -y 2 -l selfie.m -y 1
```

Understand the first and you understand compilers. The second, machines. The third, operating systems.

WHY IT IS CALLED SELFIE

Self-reference is not a party trick. It is the hard part of systems.

Every compiler you use is written in a language that some compiler had to compile. Every kernel must schedule the code that does the scheduling, and isolate itself from the very things it isolates.

Most courses step around that circle, because in a production system it is buried under a million lines. Then students learn the parts and never see the loop that ties them together.

Selfie does the opposite: it makes the loop the subject, and keeps the system small enough that the loop stays visible.

THE ROUTE

I · The system — one file, one language, fourteen instructions.
II · The three selves — compiling, executing and hosting yourself.
III · Operating systems — why they are hard, and exactly where.
IV · Proof and truth — what a machine can decide about a program.
V · Yours — what you build, and why it still matters.

The Selfie Project, University of Salzburg. Its stated purpose: *to identify and resolve self-reference in systems code, seen as the key challenge when teaching systems engineering* — hence the name.

PART I

The System

One language you can learn in an afternoon. One instruction set you can learn on the bus. And a compiler that is smaller than most build configurations.

SIZE

The whole system, in the numbers it prints about itself.

12,394
LINES · ONE FILE · SELFIE.C

661
PROCEDURES · 491 GLOBAL VARIABLES

43,492
INSTRUCTIONS GENERATED FOR ITSELF

```
$ ./selfie -c selfie.c
selfie compiling selfie.c to 64-bit RISC-U with 64-bit starc
365784 characters read in 12394 lines and 1741 comments
491 global variables, 661 procedures, 512 string literals
188392 bytes generated with 43492 instructions
```

A compiler, an emulator, a hypervisor and a library fit into 188 kilobytes of generated code and data. That is not a limitation. It is the whole pedagogical argument: a system you can finish reading is a system you can actually understand.

THE LANGUAGE · C*

7 keywords. One data type, and pointers to it.

`uint64_t` `void` `sizeof` `if` `else` `while` `return`

A strict subset of C. Five statements, the usual arithmetic and comparison operators, and the unary ∗ for dereferencing — hence the star in the name. LL(1), 22 symbols.

No structs, no arrays, no bitwise operators, no floats, no types except `uint64_t` and `uint64_t*`. You will add several of those yourself as assignments.

AND STILL UNIVERSAL

C* is Turing-complete. Anything any computer can ever compute can be written in it — clumsily, but in principle. What you learn about meaning here is *not* a scaled-down version of the real thing. It is the real thing.

WRITTEN IN ITSELF

Selfie is a C* program. The compiler that gives C* its meaning is written in the language whose meaning it gives. Hold on to that sentence — Part II is about it.

THE MACHINE · RISC-U

14 instructions. A real subset of real RISC-V.

GROUP	INSTRUCTIONS	WHAT THEY DO
initialize	lui · addi	get a constant into a register
memory	ld · sd	load and store a 64-bit word
arithmetic	add · sub · mul · divu · remu	the whole of computation
compare	sltu	is this one smaller?
control	beq · jal · jalr	branch, call, return
system	ecall	ask the world for something

32 registers, a program counter, and 4GB of byte-addressed memory. That is the entire machine model — and it is not a fiction. Selfie emits ELF binaries that run on real RISC-V hardware, on QEMU, and on the official spike emulator with the pk kernel.

FOURTEEN IS ENOUGH

No floating point. No vectors. No bitwise operations. Everything the compiler, the emulator and the hypervisor do — every one of those 43,492 instructions — is one of these fourteen.

THE PATH FROM A CHARACTER TO AN INSTRUCTION

Nothing in the middle is left out.

and in the same file: in-memory linker · disassembler · garbage collector · L1 caches · profiler · debugger with replay

selfie.c → 43,492 instructions → a running machine

Scanner and parser. Characters to symbols to a syntax tree that is never built — selfie generates code as it parses, in one pass, which is why it fits.

Code generation and linking. Registers, the stack, procedure calls, a symbol table, and an in-memory linker. Then a disassembler to read back what it wrote.

Runtime. A conservative garbage collector that even collects itself, L1 instruction and data caches, a profiler, and a debugger with replay.

PART II

The Three Selves

Self-compilation. Self-execution. Self-hosting. Three different loops — and each one closes for a different reason.

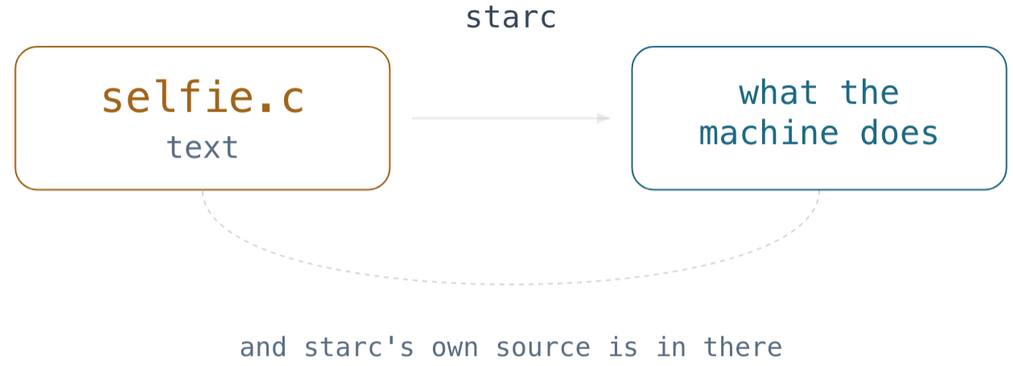
THE PROBLEM UNDERNEATH ALL THREE

A compiler defines the meaning of the language it is written in.

What does `while` mean? Not what the manual says. What it means is whatever the compiler *emits* — a comparison and a branch — and whatever the machine then does with those.

So the meaning of a program is fixed by another program. And that other program is written in the same language.

This is an English dictionary written in English. It is only paradoxical if you demand that the definitions come first. They do not: the machine comes first, and the dictionary is bootstrapped onto it.



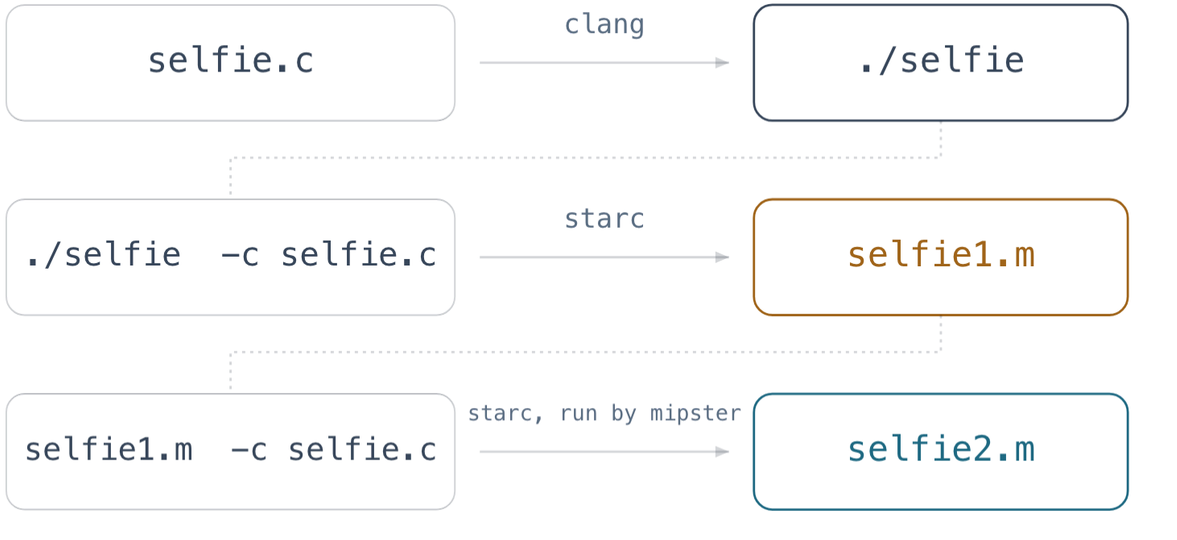
notation → meaning

TWO COLOURS, FOR THE WHOLE TALK

Ochre is syntax — text, code, notation, the thing you can check. Cyan is semantics — what actually happens, the thing you cannot.

SELF · 1 OF 3

Self-compilation: the compiler compiles its own source, and gets the same answer twice.



two roads to the same 188,392 bytes

Borrow a meaning. An ordinary C compiler builds `selfie.c` once. C* is a subset of C, so this works — and it is the only outside help the system ever gets.

Use it on itself. That binary compiles `selfie.c` to RISC-U code. Call it `selfie1.m`. Its meaning still came from the foreign compiler.

Now close the loop. *Run* `selfie1.m` and have it compile `selfie.c` again. Call that `selfie2.m` — a compiler that was compiled by itself.

`selfie1.m` and `selfie2.m` are identical. Byte for byte. That equality is called a **fixed point**, and reaching it is the moment the system stops depending on anything outside itself.

WHAT THE FIXED POINT DOES NOT BUY

The source does not determine the binary.

Ken Thompson, Turing Award lecture, 1984. Teach a compiler to recognise one particular program — say, the login program — and to insert a back door when it compiles it.

Then teach it to recognise *itself*, and to reinsert both tricks whenever it compiles its own source. Now delete both tricks from the source code.

The source is clean. Every future compiler built from that clean source carries the back door, forever, and no amount of reading finds it. Self-compilation is how it survives.

THE WAY OUT IS FROM OUTSIDE

Compile the source with a *different* compiler and compare the results — diverse double-compiling, Wheeler 2005. A system cannot certify itself; the check has to come from somewhere else.

WHAT THE FIXED POINT PROVES

That the compiler agrees with *itself* about the meaning of its own text. A real, checkable, mechanical property.

WHAT IT CANNOT PROVE

That the running binary does what the source says. That gap between what you can check and what is true is the subject of Part IV.

SELF · 2 OF 3

Self-execution: an emulator that runs its own machine code.

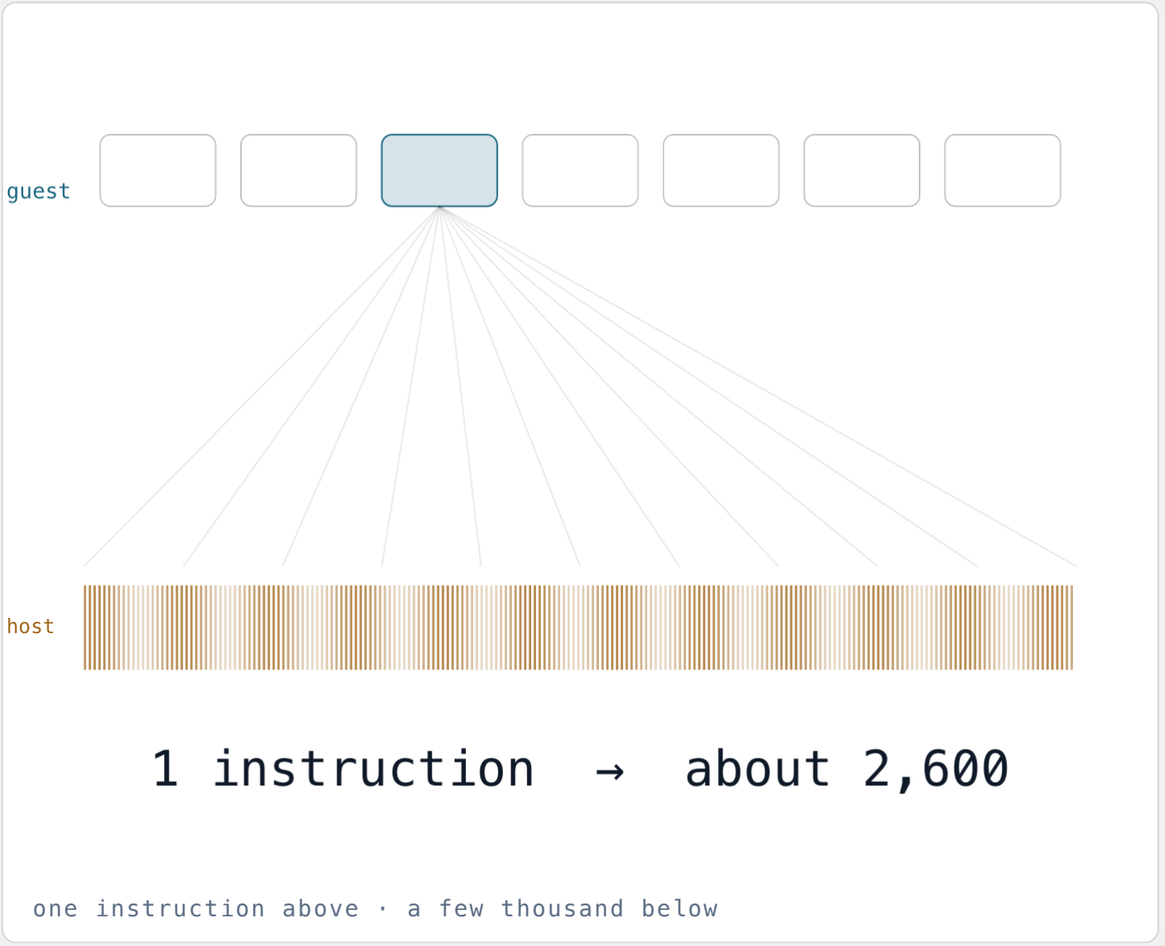
mipster is an interpreter for RISC-U, written in C*: fetch a word, decode it, do what it says, advance the program counter. About a page of code per instruction group.

Compile mipster with starc and you get RISC-U code for an interpreter of RISC-U code. Load it into mipster, and mipster is executing mipster.

The inner one cannot tell. Code has no way of asking whether the processor underneath it is silicon or software — *unless* it can see a clock, which is the one thing that gives emulation away.

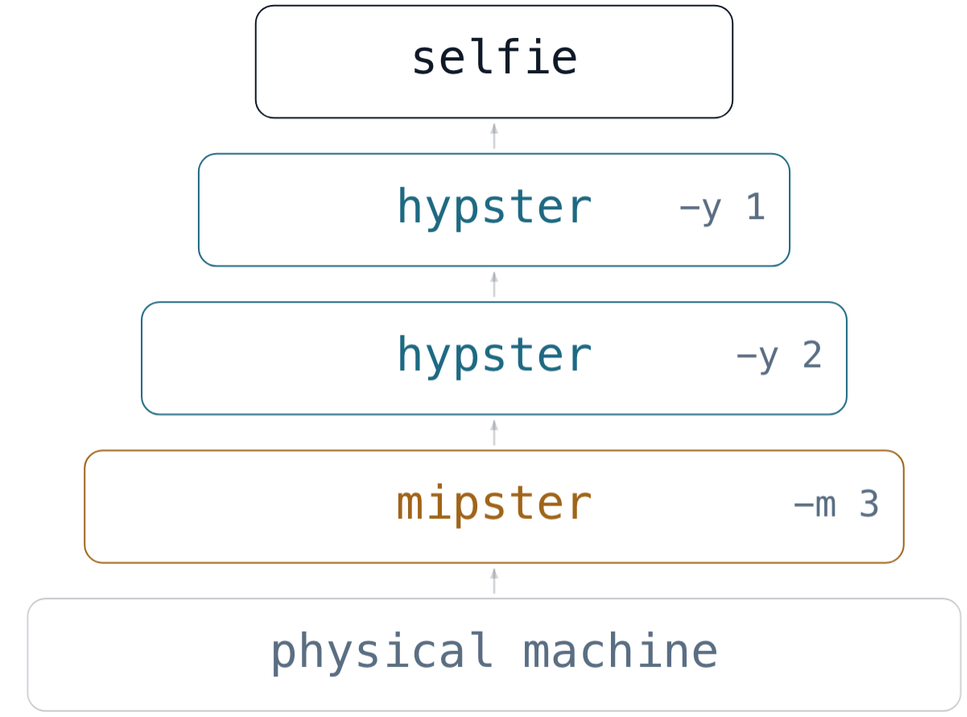
WHAT IT COSTS

One instruction up there is a few thousand down here. Selfie printing its usage line takes `85,754` instructions on the bare machine — and `222,410,917` when a mipster is emulating the mipster that runs it.



SELF · 3 OF 3

Self-hosting: a hypervisor whose virtual machines can run the hypervisor.



any stack of -m and -y · as long as it starts with mipster

hypster does not interpret anything. It creates virtual machines and then asks the machine below it to run them, by context switching: save these registers, load those, go.

Its virtual machines are good enough to host all of selfie — the compiler, the emulator, and hypster itself. Stack as many as you like, in any order.

That is what *self-hosting* means, and selfie supports it recursively, which most production hypervisors do not.

ONE RULE

Every tower has to stand on a mipster. Context switching has to come from somewhere, and stock RISC-V hardware does not offer it in the form hypster needs — so the bottom of the stack is always an emulated machine.

PART III

Why Operating Systems Are Hard

This is the part of the talk I would keep if I only had five minutes. It is one distinction, and it is missing from almost every course on the subject.

WHAT A KERNEL OWES YOU

Three problems. Two are ordinary. One is not.

ISOLATION IN SPACE

Nobody may read or write anyone else's memory. Solved by paging: every machine sees a 4GB address space of its own, and a page table maps it onto whatever physical frames are free.

ISOLATION IN TIME

Everybody eventually runs. Solved by a timer and a scheduler: preempt, save the registers, pick someone else, restore, go.

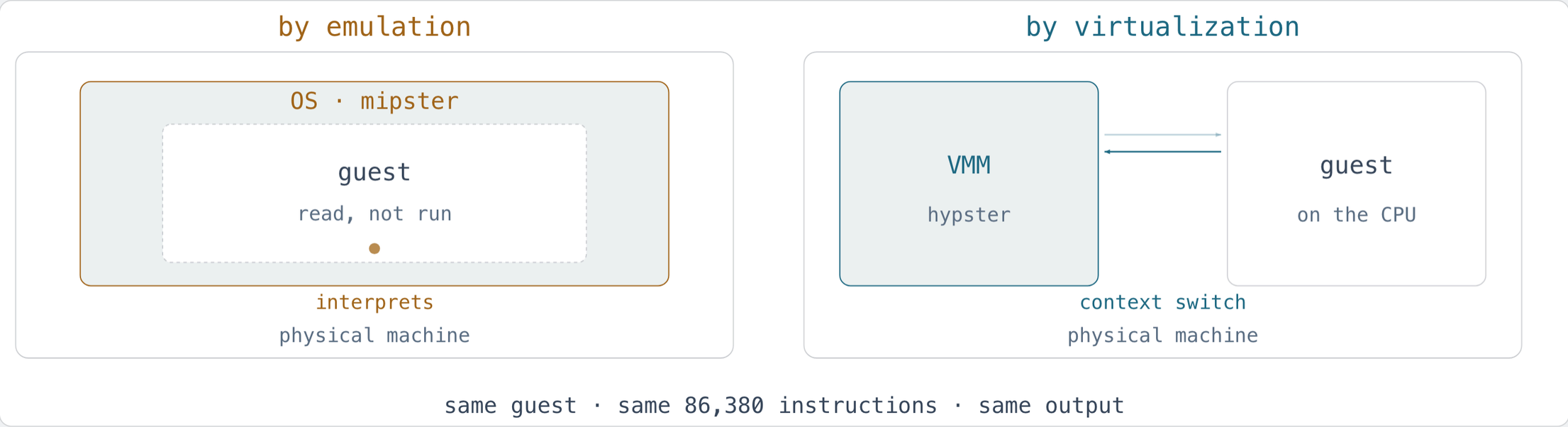
AND THE KERNEL ITSELF

The code that provides isolation is also code, and it also runs on the machine. It has to be isolated from the things it isolates. By what?

The first two are engineering: difficult, well understood, teachable in a fortnight. The third is self-reference — and it is where the intuition breaks.

THE DISTINCTION

Two ways to build the same operating system.



By emulation. The OS contains an interpreter. Guest instructions are read and carried out by OS code; guest code never touches the processor. Isolation is *free* — the guest has no way to reach anything, because it is only ever data being read.

By virtualization. The OS sets up a page table and a timer and then lets the guest run *on the real processor*, taking control back on faults, calls and interrupts. Isolation now has to be constructed, because the guest and the kernel share the same hardware.

THE CLAIM

An OS by emulation is semantically equivalent to an OS by virtualization.

Not similar. Equivalent — down to every bit the guest can observe. There is no experiment the guest can run that distinguishes the two, except by looking at a clock.

Here is the evidence, on this laptop. Selfie hosted by an OS built each way, printing its usage line:

```
$ make emu-emu // OS by emulation
guest: 86,380 executed instructions
$ make os-emu // OS by virtualization
guest: 86,380 executed instructions
```

The same guest, doing the same work, instruction for instruction. Selfie even ships a procedure called `mixter` that switches a running machine between the two *mid-execution*, and nothing notices.

SO EMULATION IS AN EXECUTABLE SPECIFICATION

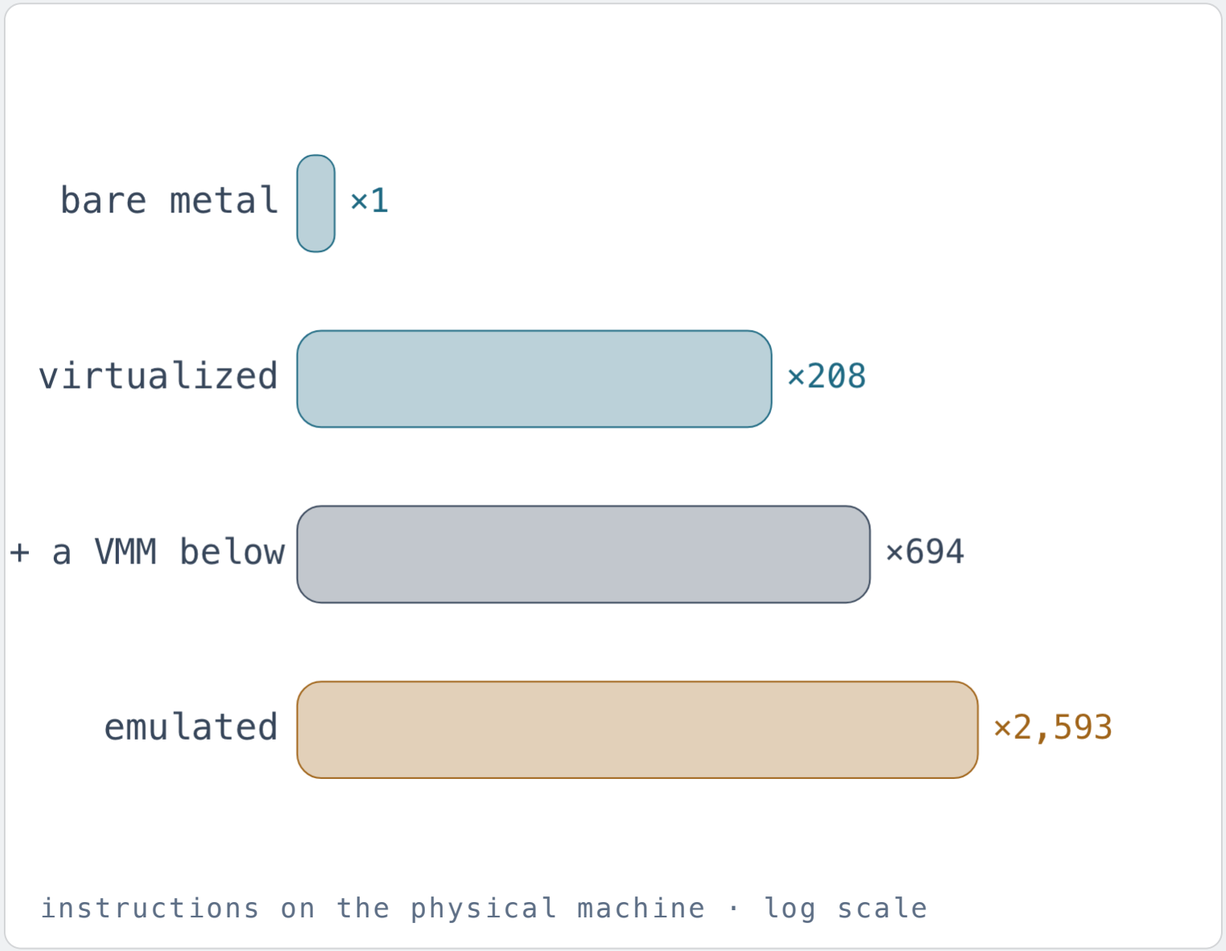
If the two are equivalent, the simple one *defines* what the complicated one must do. Build the emulated version first, and you have a running answer key for the virtualized one.

THIS IS NOT JUST A TEACHING TRICK

It is how parts of real operating systems get tested and verified: an interpreter as the reference, and the fast implementation checked against it.

SO WHY VIRTUALIZE AT ALL?

For exactly one reason: performance.



Same guest. Same output. Same 86,380 instructions of actual work. The bill sent to the machine underneath:

bare metal	85,754	×1
virtualized	17,860,937	×208
+ a VMM below	59,492,501	×694
emulated	222,410,917	×2,593

Virtualization wins by a factor of 12 here — and by much more on real work, because the cost of a context switch is fixed and gets amortised. Give the guest something substantial to do, say self-compiling, and the overhead falls from ×208 to ×1.7. Production systems get close to ×1. *That* is why the cloud exists.

Virtualization is emulation plus self-reference.

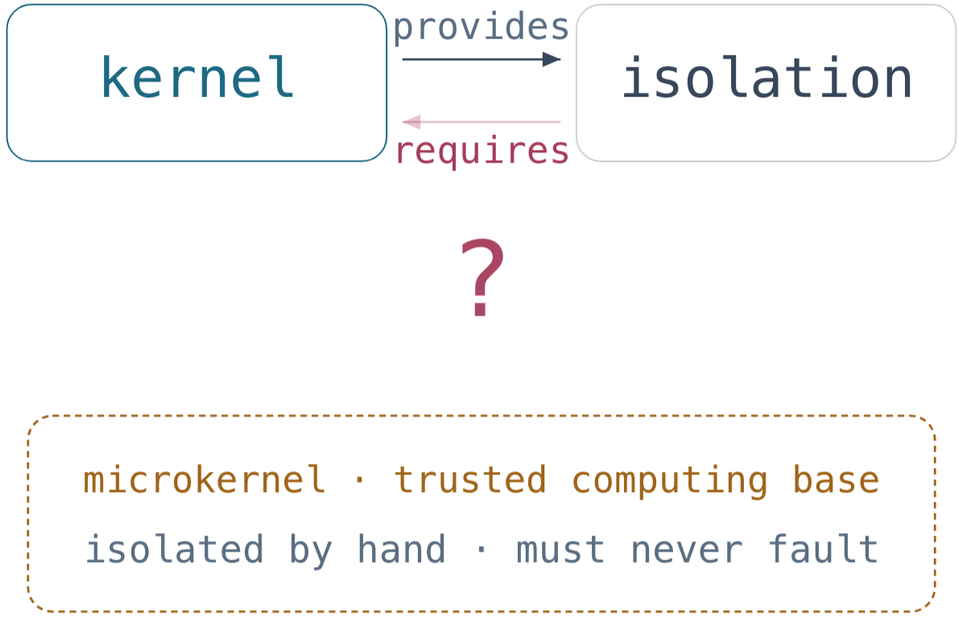
The kernel now runs on the same processor as its guests. To be isolated from them it must itself run in a virtual machine. And who manages that one?

That is the bootstrapping problem, and it is where the difficulty of operating systems actually lives. Real systems answer it by writing kernel code that carefully *never uses* the abstractions it provides: it must not fault, because it is what handles faults.

What is left when you shrink that code to the minimum is a microkernel — the trusted base everything else stands on. Small enough that people have proved it correct.

EMULATION IS ISOLATION. VIRTUALIZATION IS ISOLATION *PLUS* SELF-REFERENCE.

That one line is the summary of this Part. In an emulator there is no self-reference at all: the guest is data, and data cannot escape.



THE PAYOFF

Take the self-reference out. Put it back on purpose.

WHY THE SUBJECT IS HARD

Because the standard presentation shows you a kernel that lives inside the abstraction it implements, and never separates that from the ordinary problems of memory and time. Everything is tangled with everything, so nothing can be understood on its own.

WHY IT STOPS BEING HARD

Build the operating system by emulation first. Now isolation is trivial and only space and time are left. Then swap in virtualization, which changes no behaviour and buys only speed — and the *only* thing that got harder is the self-reference. Now you can see it, name it, and study it alone.

Two designs, one semantics, one difference. That difference is the subject — and it is the thing nobody told me when I was a student.

PART IV

Proof and Truth

Everything so far was construction. Now the other half: what can a machine decide about a program — and what can it never decide?

THE GAP

Syntax you can check. Semantics you cannot.

DECIDABLE · ABOUT THE TEXT

Does it parse? How long is it? Which procedures does it call? Are the types consistent? Does it contain a loop? A compiler answers all of these, always, in finite time.

UNDECIDABLE · ABOUT THE MEANING

Does it terminate? Is it equivalent to that one? Can it divide by zero? Can it read outside its memory? Is it correct? No program decides any of these for all programs.

RICE · 1953

> *Every non-trivial property of the behaviour of programs is undecidable. Only questions about their text are safe.*

So every tool that says anything useful about behaviour is a deliberate approximation: sound but incomplete, or complete but unsound, or exact only within a bound. Choosing which to give up *is* the discipline — and selfie ships four different choices for you to compare.

THE EXTRAS · TURNING EXECUTION INTO LOGIC

Machine code in. Formulae out. Solver next.

monster	symbolic execution of RISC-U into SMT-LIB — satisfiable exactly when some input makes the code exit non-zero or divide by zero
beator	the same idea as bounded model checking, into BTOR2 — adds memory access outside allocated blocks
rotor	full RISC-V, not just RISC-U, into BTOR2 *and* SMT-LIB — and models that let you *synthesise* code rather than only check it
bitme	a concurrent bounded model checker over rotor's models, driving SMT solvers and binary decision diagrams
beatle	draws BTOR2 formulae as graphs, so you can look at what your program means
rotor-rust	rotor in Rust, with a browser visualiser and symbolic command-line arguments

ALL OF THEM ARE SELF-APPLICABLE

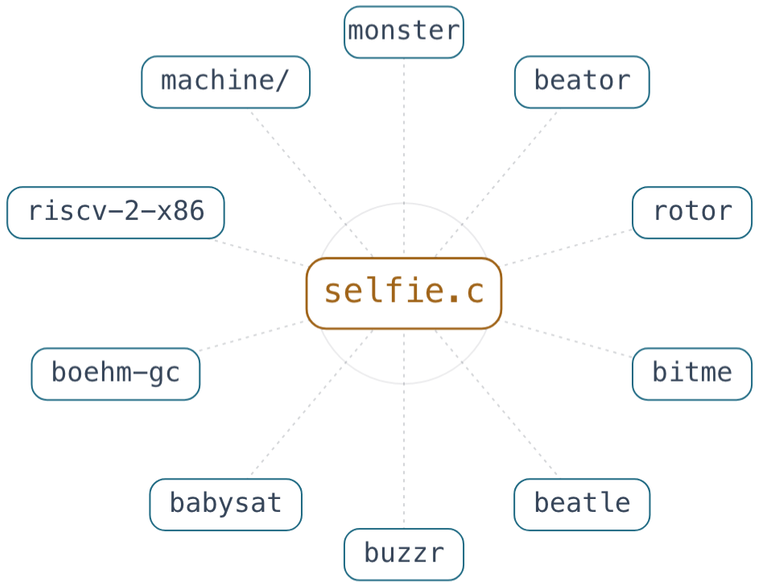
Each translates RISC-U or RISC-V code *including all of selfie and itself*. You can hand a symbolic execution engine its own machine code and ask a solver about it.

AND THEN YOU HIT THE WALL

The formulae are satisfiable or not — and deciding that is NP-complete. You can watch syntax become semantics and run into the limits of computation before the coffee gets cold.

THE REST OF THE WORKSHOP

Everything else that grew out of one file.



built on selfie · applied to selfie

buzzr — a fuzzer that fuzzes RISC-U code including all of selfie and itself. A self-fuzzing fuzzer.

babysat — a brute-force SAT solver over DIMACS CNF, so the solver at the end of the previous slide is not a black box either.

boehm-gc — an O(n) collector for small blocks beside selfie's own conservative, self-collecting $O(n^2)$ one. Two garbage collectors to compare, in one system.

riscv-2-x86 — a self-translating binary translator: RISC-U code, including selfie and itself, into x86.

machine/ — bare metal. No emulator, no operating system, selfie on a real RISC-V board.

Plus a benchmark suite, a docker image, a browser you can run the whole thing in, and a shelf of student theses that started exactly where you are sitting.

PART V

Yours

What you actually do in the classes — and the honest answer to the question you are all sitting there wanting to ask.

THE WORK

You do not read the system. You change it.

COMPILER CLASS

Hexadecimal literals. Bitwise and logical operators. `for` loops. Lazy evaluation. Arrays, then multidimensional arrays. Structs — declaration, then execution. Each one is a change to the scanner, the parser and the code generator at once.

SYSTEMS CLASS

An assembler that parses selfie's own assembly, then assembles itself. Processes. `fork`, `wait`, `exit`. Locks. Threads. A thread-safe allocator. A Treiber stack. You are writing the operating system, not reading about one.

AND ONE RULE ABOVE ALL OF THEM

Whatever you change, selfie must still compile itself. Your new code generator has to be good enough to generate the compiler that contains it.

That rule is the most demanding code reviewer you will ever have, and it never sleeps. It is also why the bugs you write teach you something instead of just costing you marks.

```
$ ./grader/self.py self-compile
grade: 2
```

The autograder is public. You grade yourself before you submit — and you will know your grade before I do.

THE QUESTION EVERYONE IS HOLDING

"Why learn this when a machine will write the code for me?"

GENERATION GOT CHEAP. VERIFICATION DID NOT.

Producing plausible code is now nearly free. Deciding whether it is *right* is exactly as hard as it was in 1953, and Rice's theorem does not care who wrote the program. Value moves to the two ends: saying precisely what should be true, and establishing that it is.

THE STACK DID NOT DISAPPEAR.

Every model that writes your code runs on virtualized machines, compiled by a compiler, scheduled by a kernel, on a processor executing instructions. When it is slow, or wrong, or leaking, the person who can go down there is the person who fixes it.

DEPTH IS NOT DOWNLOADABLE.

Knowing where a field is thin, where it is wrong, and where it is about to give — that comes from having lived inside a subject. No summary transfers it. Selfie is small enough that you can live inside *all* of it.

The frontier is not proving things inside a language — machines are formidable at that. It is finding the truth you cannot yet prove, and building the language that captures it.

Which is the argument of a companion lecture, if you want the long version: `selfie.cs.uni-salzburg.at/intelligence`

TAKE A SELFIE

Fifteen minutes from now you could be running a compiler that compiles itself.

```
$ git clone github.com/cksystemsteaching/selfie
$ cd selfie && make
$ ./selfie -c selfie.c -m 2 -c selfie.c
```

No C compiler? `docker run -it cksystemsteaching/selfie`. No terminal at all? It runs in a browser tab.

Then open `selfie.c`, find the last line before `exit_code`, and print your own name. That is assignment one, and it is genuinely how the course starts.

Book — *Elementary Computer Science: From Bits and Bytes to the Universality of Computing*

Slides, autograder, assignments — all public, all in the repository

Slack — the `#selfie` channel, where the arguing happens

`selfie.cs.uni-salzburg.at`

`github.com/cksystemsteaching/selfie`